



Cyberscope

A *TAC Security* Company

Audit Report

Pool Games

April 2026

Repository : <https://github.com/AllexAntoniollo/PoolGames>

Commit : b2b39a68f70a4358d35d56d7146ca3c1e03f3272

Audited by © cyberscope

Table of Contents

Table of Contents	1
Risk Classification	4
Review	5
Audit Updates	5
Source Files	5
Contract Readability Comment	6
Overview	7
PaymentPool	7
FeeManager	7
PoolGamesNft	8
TreasuryPool	8
UserPoolGames	9
Findings Breakdown	10
Diagnostics	11
CCD - Claimed Contribution Discrepancy	13
Description	13
Recommendation	14
IRI - Inconsistent Recipient Identification	15
Description	15
Recommendation	15
IDR - Integer Division Revert	16
Description	16
Recommendation	17
RPC - Repeated Pool Claim	18
Description	18
Recommendation	19
SPD - Sybil Pool Drain	20
Description	20
Recommendation	24
TSI - Tokens Sufficiency Insurance	25
Description	25
Recommendation	25
DTV - Duplicate Token Validation	26
Description	26
Recommendation	26
IRV - Inconsist Recipient Validation	27
Description	27
Recommendation	28
OCTD - Transfers Contract's Tokens	29

Description	29
Recommendation	30
PIF - Potential Initialization Front-Run	32
Description	32
Recommendation	32
AAO - Accumulated Amount Overflow	33
Description	33
Recommendation	33
ALM - Array Length Mismatch	34
Description	34
Recommendation	35
CCR - Contract Centralization Risk	36
Description	36
Recommendation	37
DDP - Decimal Division Precision	38
Description	38
Recommendation	39
ERP - ERC777 Reentrancy Path	40
Description	40
Recommendation	41
GO - Gas Optimization	42
Description	42
Recommendation	42
HV - Hardcoded Values	43
Description	43
Recommendation	44
MC - Missing Check	45
Description	45
Recommendation	45
MEM - Missing Error Messages	46
Description	46
Recommendation	46
MEE - Missing Events Emission	47
Description	47
Recommendation	48
PTAI - Potential Transfer Amount Inconsistency	49
Description	49
Recommendation	50
PTRP - Potential Transfer Revert Propagation	51
Description	51
Recommendation	52
SDOT - Stale Data on Transfer	53

Description	53
Recommendation	54
SMUS - Stale Migration User State	55
Description	55
Recommendation	55
SRC - Sybil Referral Chain	56
Description	56
Recommendation	56
L04 - Conformance to Solidity Naming Conventions	57
Description	57
Recommendation	58
L07 - Missing Events Arithmetic	59
Description	59
Recommendation	59
L16 - Validate Variable Setters	60
Description	60
Recommendation	60
L22 - Potential Locked Ether	61
Description	61
Recommendation	61
Functions Analysis	62
Summary	67
Disclaimer	68
About Cyberscope	69

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	24 Mar 2026
---------------	-------------

Source Files

Filename	SHA256
UserPoolGames.sol	fe3dc59c4f45152046a9ac71cf4582feae165a47d303dfa93c4f914007f25a8
TreasuryPool.sol	6c0fb05b0314b3a5b8e86c8c0e8dbe5dd36ff285fa0d1f69bd09ec448fa17fdc
PoolGamesNft.sol	5ce59e07b644feb4f108cf6d38882dee62a175307a85f3526e2906a5fd1788f5
PaymentPool.sol	1f38006748f16bd55d40549b8b28964e79c039941eeab35dfd47848f22948698
IUserPoolGames.sol	3adc9d862abc7213f1ff3e565f5b4b8164165021b9e9b9c7be5bef148b10b51a
ITreasuryPool.sol	3c31580aab6cee5ababda816c2f4709b2eced5f28647862e0a8ad1f42e5db3b
IManager.sol	cf616d117a6a014406cde0a1f0df64ca945979a6016850ab5fd3289658aa682b
FeeManager.sol	1ca4a6dae2fb02b250bfe934f6a98ec9f36368d89b7c7e2760c3e8aeaf081f35

Contract Readability Comment

The audit scope is to check for security vulnerabilities, validate the business logic and propose potential optimizations. The contracts exhibit several deviations from established Solidity best practices regarding code readability, economic design, state management, and architectural consistency. Given these issues, it is not advisable to assume that the contracts are in a production-ready state. The development team is strongly encouraged to re-evaluate the business logic, economic model, and Solidity guidelines to ensure that the contracts adhere to established best practices and security measures. The economic model should be reviewed to ensure the sustainability of the protocol and the Solidity implementation should be strengthened to guarantee the security of the funds across all operational flows.

Overview

The PoolGames platform is a referral-based investment protocol built around a stablecoin token. Users register under sponsors to form a 20-level referral tree, contribute funds on fixed-term plans to earn returns, and purchase NFTs that entitle them to dividend distributions. The protocol operates across four core contracts interconnected through a circular dependency: `TreasuryPool` holds user deposits and manages contribution lifecycles, `UserPoolGames` maintains the referral tree and distributes commissions, `PoolGamesNft` handles NFT sales and dividend payouts, and `FeeManager` collects platform fees from all three. Three of the four contracts are UUPS-upgradeable proxies controlled by a single owner address.

PaymentPool

`PaymentPool` is an abstract contract that serves as the base for fee collection and distribution. It maintains a list of registered ERC20 tokens and a set of recipients, each assigned a percentage share denominated in parts-per-million. When funds are deposited via `incrementBalance`, they accumulate in an undistributed pool. When any recipient calls `claim`, the contract distributes the undistributed balance across all recipients according to their percentages and transfers the caller's accumulated share. The contract supports multi-token accounting, tracking claimable balances per recipient per token. The owner manages tokens and recipients, can mark wallets as immutable to lock their percentage, and has the ability to withdraw unclaimed balances or sweep the full token balance of the contract.

FeeManager

Feemanager is the deployment of `PaymentPool`. It inherits all fee collection and distribution logic and serves as the central fee recipient for the platform. Revenue from NFT sales, contribution cancellation penalties, and excess unilevel commissions all flow into `FeeManager` via `incrementBalance`, where they are distributed to configured recipients. It uses Ownable2Step for safer two-step ownership transfer.

PoolGamesNft

PoolGamesNft is a UUPS-upgradeable ERC721 contract that manages the platform's NFT collection. Users purchase NFTs at tiered prices based on token ID ranges, with revenue split across three destinations: 50% to the old pool, 30% to the new pool, and 20% to FeeManager . NFT holders are entitled to dividend distributions funded through addValueToDistribute . Distributions are processed in batches via processPayments , which operates as a two-phase state machine: the first phase pays existing NFT holders their equal share of the distribution pool, and the second phase migrates any NFTs purchased during the distribution period into the active set for future rounds. The contract accepts native ETH through receive and fallback functions.

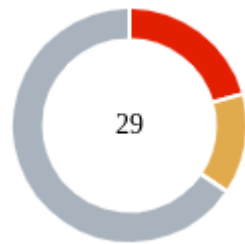
TreasuryPool

TreasuryPool is a UUPS-upgradeable contract where registered users deposit tokens on fixed-term plans ranging from 1 to 360 days, with promised returns ranging from 0.1% to 240%. Upon contribution, 75% of the promised profit is immediately distributed as referral commissions through the UserPoolGames unilevel system. Users claim their profit in 30-day intervals over the plan duration, with the original deposit returned upon completion. A cancellation mechanism allows users to exit early in exchange for a 50% penalty and a 20% fee sent to FeeManager. The contract tracks each user's contributions individually but maintains shared per-user counters for total expected profit and accumulated unilevel earnings, which are used to offset claim payouts against commissions already received. A bot wallet address can execute claims on behalf of any user. Critical contract dependencies including the user contract, fee manager, and bot wallet are configurable by the owner.

UserPoolGames

UserPoolGames is a UUPS-upgradeable contract that manages user registration and the 20-level referral tree. Anyone can register by providing a registered sponsor, which builds a chain of up to 20 ancestor levels. When a user contributes to `TreasuryPool`, the `distributeUnilevel` function iterates through the contributor's referral chain and distributes commission shares to each qualifying ancestor. Qualification at each level requires a minimum number of valid direct referrals and a minimum amount deposited in `TreasuryPool`. Ancestors who do not meet the requirements for their level have their share redirected to `FeeManager` as excess. The owner can batch-create users for migration purposes and configure the treasury pool and fee manager addresses.

Findings Breakdown



● Critical	6
● Medium	4
● Minor / Informative	19

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	6	0	0	0
● Medium	4	0	0	0
● Minor / Informative	19	0	0	0

Diagnostics

● Critical
 ● Medium
 ● Minor / Informative

Severity	Code	Description	Status
●	CCD	Claimed Contribution Discrepancy	Unresolved
●	IRI	Inconsistent Recipient Identification	Unresolved
●	IDR	Integer Division Revert	Unresolved
●	RPC	Repeated Pool Claim	Unresolved
●	SPD	Sybil Pool Drain	Unresolved
●	TSI	Tokens Sufficiency Insurance	Unresolved
●	DTV	Duplicate Token Validation	Unresolved
●	IRV	Inconsist Recipient Validation	Unresolved
●	OCTD	Transfers Contract's Tokens	Unresolved
●	PIF	Potential Initialization Front-Run	Unresolved
●	AAO	Accumulated Amount Overflow	Unresolved
●	ALM	Array Length Mismatch	Unresolved
●	CCR	Contract Centralization Risk	Unresolved
●	DDP	Decimal Division Precision	Unresolved

●	ERP	ERC777 Reentrancy Path	Unresolved
●	GO	Gas Optimization	Unresolved
●	HV	Hardcoded Values	Unresolved
●	MC	Missing Check	Unresolved
●	MEM	Missing Error Messages	Unresolved
●	MEE	Missing Events Emission	Unresolved
●	PTAI	Potential Transfer Amount Inconsistency	Unresolved
●	PTRP	Potential Transfer Revert Propagation	Unresolved
●	SDOT	Stale Data on Transfer	Unresolved
●	SMUS	Stale Migration User State	Unresolved
●	SRC	Sybil Referral Chain	Unresolved
●	L04	Conformance to Solidity Naming Conventions	Unresolved
●	L07	Missing Events Arithmetic	Unresolved
●	L16	Validate Variable Setters	Unresolved
●	L22	Potential Locked Ether	Unresolved

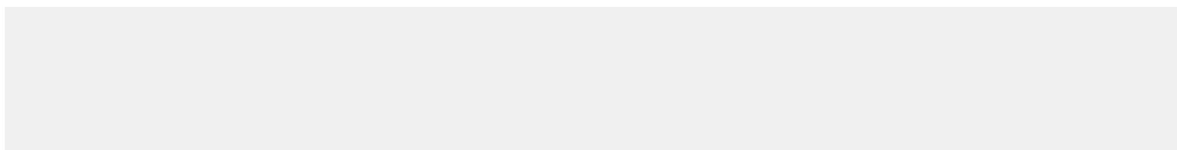
CCD - Claimed Contribution Discrepancy

Criticality	Critical
Location	TreasuryPool.sol#L273
Status	Unresolved

Description

The `cancelContribution` function currently only checks if `daysPaid` is less than `maxPeriod` before allowing a cancellation. This approach permits the cancellation of contributions that have already been partially claimed. When a cancellation occurs, the function subtracts the entire original profit from the global per-user `totalProfitToClaim` counter. This subtraction does not account for any prior `claimContribution` calls that have already reduced this counter by a portion of the original profit. Since `totalProfitToClaim` is a shared counter across all of a user's contributions and is not tracked individually per contribution, this results in a double subtraction for a portion of the profit. Consequently, this error can corrupt the accounting for all other active contributions held by the user, potentially leading to locked funds from different contributions.

Furthermore, `cancelContribution` does not adjust the `totalUnilevelProfit` counter. When a contribution is cancelled, the profit associated with that contribution is removed from `totalProfitToClaim`, but the unilevel earnings the user already received remain unchanged. This can cause `totalUnilevelProfit` to exceed `totalProfitToClaim`, which means that during future `claimContribution` calls on the user's remaining contributions, the `totalUnilevelProfit >= totalValueInUSD` branch will be taken, setting `totalValueInUSD = 0` and causing the pool to pay nothing. The user effectively loses legitimate profit from their other active contributions because the system believes those profits were already covered by unilevel earnings that were, in reality, associated with the now-cancelled contribution.



Shell

```
uint profit = donation.balance - donation.deposit;  
if (totalProfitToClaim[msg.sender] >= profit) {  
    totalProfitToClaim[msg.sender] -= profit;
```

Shell

```
if (totalUnilevelProfit[user] >= totalValueInUSD) {  
    totalUnilevelProfit[user] -= totalValueInUSD;  
    totalProfitToClaim[user] -= totalValueInUSD;  
    totalValueInUSD = 0;  
} else {  
    totalProfitToClaim[user] -= totalValueInUSD;  
    totalValueInUSD -= totalUnilevelProfit[user];  
    totalUnilevelProfit[user] = 0;  
}
```

Recommendation

The team could consider to subtract only the unclaimed portion upon cancellation or to prevent cancellation once any claims have been made on a contribution. This prevents cancellations if any part of the contribution has already been claimed, ensuring the integrity of the claims process. Additionally, the `cancelContribution` function should reduce `totalUnilevelProfit` proportionally to the cancelled contribution's profit, ensuring it does not exceed the updated `totalProfitToClaim`. This prevents future claims on remaining contributions from being incorrectly zeroed out due to stale unilevel accounting.

IRI - Inconsistent Recipient Identification

Criticality	Critical
Location	PaymentPool.sol#L158
Status	Unresolved

Description

In the `PaymentPool.sol` contract, `isRegistered[msg.sender] = true` incorrectly marks the owner's address as registered instead of the intended `newRecipient`. This inconsistency allows the same recipient to be added multiple times to the `recipients[]` array through repeated calls. Each duplicate entry results in an additional percentage allocation during the distribution loop in the `claim()` function.

Shell

```
isRegistered[msg.sender] = true
```

Recommendation

The team is advised to ensure the `isRegistered` flag correctly applies to the intended recipient.

IDR - Integer Division Revert

Criticality	Critical
Location	PoolGamesNft.sol#L203,221
Status	Unresolved

Description

In the `distributePayment()` function, each NFT holder receives a share calculated as `valueToDistribute / totalWeightNftToReceivePayment`, which uses integer division. After distributing to all N NFT holders, the total value paid, `valuePaidInDistribution`, equals $N * \text{floor}(V/N)$. Consequently, any remainder from the division, $V \% N$, is not distributed.

In the `processPendingIds()` function, a check is performed:

```
Shell
if (valuePaidInDistribution < valueToDistribute)
    revert("Not all value distributed");
```

This condition triggers a revert whenever the total value to distribute, is not perfectly divisible by N . As a result, the contract's state machine becomes permanently locked: `isDistributePeriod` remains true, `valueToDistribute` is never reset, and no new distributions can commence.

This leads to a permanent denial-of-service (DoS) state where all accumulated funds are irretrievably locked.

Furthermore, the `addValueToDistribute()` function at line 107 lacks access control, allowing anyone to inject minimal amounts to ensure the revert condition is met, exacerbating the issue.

Recommendation

To address the issue the team is advised to replace the strict less-than check with a tolerance for minor discrepancies. This can be achieved by computing the last holder's share as the difference between `valueToDistribute` and `valuePaidInDistribution`. This adjustment ensures that any remaining value is absorbed by the last holder, maintaining the integrity of the distribution.

Additionally, the team is advised to implement access control for the `addValueToDistribute()` function. By restricting access to this function, unauthorized modifications can be prevented, enhancing the security and reliability of the system.

RPC - Repeated Pool Claim

Criticality	Critical
Location	PaymentPool.sol#L91,110,115
Status	Unresolved

Description

When the total percentage is less than the defined precision, each invocation of the `claim()` function distributes only a portion of the `balanceFree[token]` to recipients, leaving some funds undistributed. Since `balanceFree[token]` is recalculated at the beginning of each `claim()` call, subsequent calls continue to distribute fractions of the remaining balance. This process allows a recipient to deplete the entire `balanceFree` balance through repeated calls, following a pattern of geometric decay.

A single recipient can exploit this mechanism across multiple transactions to deplete the pool.

```
Shell
uint amount = balanceFree[token];
...
uint value = (amount * recipientsPercentage[user])
/ PRECISION;
recipientsClaim[user][token] += value;

balanceFree[token] -= value;
```

Recommendation

The team should ensure that the distribution mechanism accounts for the unallocated portion of `balanceFree`. After the distribution loop, the remaining balance should either be zeroed out or explicitly locked to prevent repeated redistribution.

SPD - Sybil Pool Drain

Criticality	Critical
Location	TreasuryPool.sol PoolGamesNft.sol
Status	Unresolved

Description

An attacker can drain the TreasuryPool by combining three design flaws: (1) referral chain creation is permissionless via `createUser()`, (2) commissions are paid immediately at contribution time based on the promised profit, not the deposit amount and (3) `cancelContribution()` allows immediate cancellation with a 50% refund. The attacker creates a small sybil referral chain, contributes, collects commissions to their own sybil accounts, immediately cancels for a 50% refund, and repeats, compounding profits each cycle.

For the 360-day plan, a 1,000 USDC deposit generates 2,400 USDC in promised profit, of which 28% (672 USDC) flows to the attacker's sybil chain at levels 1-2. After cancellation, the attacker receives a 500 USDC refund. Total received (1,172) exceeds the deposit (1,000), yielding 172 USDC profit per cycle. Since `cancelContribution()` decreases `valueInPool` and the attacker's sybil accounts remain qualified, the attacker can contribute again immediately with the full proceeds.

Shell

```
usdc.approve(address(userContract), (profit * 75) / 100);  
  
userContract.distributeUnilevel(msg.sender,  
profit);
```

Shell

```
valueInPool[msg.sender] -= donation.deposit;
```

Shell

```
function createUser(address _sponsor) external  
nonReentrant {  
    require(!users[msg.sender].registered,  
        "Already Registered");  
    require(users[_sponsor].registered, "Invalid  
Sponsor");
```

A detailed example follows:

Shell

Setup (one-time, 400 USDC):

Attacker creates 4 accounts: A (root), B, D, and C.

- B calls `createUser(A)`, D calls `createUser(A)`, C calls `createUser(B)`.

- C's referral chain: `levels[0] = B, levels[1] = A`.

- A contributes 200 USDC (meets level 2 `valueInPool` requirement of 200).

- B contributes 100 USDC → becomes valid → `A.directs = 1`.

- D contributes 100 USDC → becomes valid → `A.directs = 2` (level 2 needs 2).

- Qualification met: B has 1 direct + 100 in pool (level 1). A has 2 directs + 200 in pool (level 2).

Shell

Cycle 1: C contributes 1,000 USDC (360-day plan)

- `profit = (1000 \* 2400) / 1000 = 2400`

- `distributeUnilevel` pays: B = `(2400 \* 20) / 100 = 480`, A = `(2400 \* 8) / 100 = 192`. Total to attacker: 672 USDC.

- C cancels immediately: `refund = 500`, `fee = 100`. `valueInPool[C]` resets to 0.

- **Cycle P&L: spent 1,000, received 672 + 500 = 1,172. Profit: +172. Cash: 1,172.**

Cycle 2: C reinvests 1,172 USDC
- `profit = 2812`. B gets 562, A gets 224 = 786 to attacker.
- Cancel: refund 586, fee 117.
- **Profit: +200. Cash: 1,372.**

Cycle 3: C reinvests 1,372 USDC
- `profit = 3292`. B gets 658, A gets 263 = 921 to attacker.
- Cancel: refund 686, fee 137.
- **Profit: +235. Cash: 1,607.**

Cycle 4: C reinvests 1,607 USDC
- B gets 771, A gets 308 = 1,079.
- Cancel: refund 803, fee 160.
- **Profit: +275. Cash: 1,882.**

Cycle 5: C reinvests 1,882 USDC
- B gets 903, A gets 361 = 1,264.
- Cancel: refund 941, fee 188.
- **Profit: +323. Cash: 2,205.**

After 5 cycles: attacker started with 1,400 USDC (1,000 + 400 setup), now holds 2,205 liquid + 400 locked = 2,605 total, 86% ROI. Pool lost ~9,845 USDC. Profits compound ~17% per cycle. By cycle ~20 the attacker hits `MAX_ROOF` (10,000 USDC per deposit), extracting 1,720 USDC per cycle indefinitely.

Recommendation

The team should follow best practices to enhance the system's security and efficiency. Such practices include cooldown periods, delayed commission payments, solvency checks on payouts, and consistent economic models. These measures help prevent potential abuse of the system.

TSI - Tokens Sufficiency Insurance

Criticality	Critical
Location	TreasuryPool.sol
Status	Unresolved

Description

The contract lacks mechanisms to ensure the sustainability of its token supply. It operates by funding returns solely through new deposits. Each new deposit is guaranteed a return that exceeds the initial amount, with 75% of the promised profit being immediately allocated as referral commissions to current members. Consequently, the contract's financial obligations consistently surpass its available assets, making its solvency reliant on the continuous influx of new deposits.

Shell

```
profit = (amount * config.profitPercent) / 1000;  
uint totalReturn = amount + profit;
```

Recommendation

The team should ensure the contract's obligations are always covered by its reserves. This will ensure users will always be able to withdraw their funds.

DTV - Duplicate Token Validation

Criticality	Medium
Location	PaymentPool.sol#L63
Status	Unresolved

Description

The `addToken` function allows a token to be added to the `tokens` array without verifying if it is already present. This inconsistency can lead to the same token being added multiple times. Consequently, when the `claim` function is executed, it iterates over all entries in the `tokens` array and processes the same token multiple times. This results in inflated payouts that exceed the actual deposits.

Shell

```
function addToken(address token, string calldata name)
external onlyOwner {
    require(token != address(0), "Token address
cannot be zero");
    require(bytes(name).length > 0, "Token name
cannot be empty");
    tokenNames[token] = name;
    tokens.push(IERC20(token));
}
```

Recommendation

The team could incorporate the `require(!isValidToken(token))` validation in the `addToken` method. This would ensure duplicate tokens are not processed.

IRV - Inconsist Recipient Validation

Criticality	Medium
Location	PaymentPool.sol#L187
Status	Unresolved

Description

In the `PaymentPool.sol` contract, the code uses the check `require(isRegistered[msg.sender])` to validate the registration status of the caller, who is the owner, instead of the intended recipient parameter. . Consequently, this check will always succeed, assuming the owner is registered. This can lead to an inconsistent state within the contract.

Shell

```
function updateRecipientPercentage(
    address recipient,
    uint24 newPercentage
) external onlyOwner {
    require(isRegistered[msg.sender], "Recipient does not exist");
    require(
        !immutableWallets[recipient],
        "This recipient is immutable and cannot have their percentage updated"
    );
    ...
}
```

Recommendation

The team is advised to ensure the targeted recipient is the entity checked against registration.

OCTD - Transfers Contract's Tokens

Criticality	Medium
Location	PaymentPool.sol#L219,224
Status	Unresolved

Description

The contract grants the owner the ability to claim the entire balance held within it. This is executed through the `sweepTokens` function, which retrieves the total token balance of the contract using `IERC20(token).balanceOf(address(this))` and transfers it entirely to the owner. This action includes both undistributed deposits (`balanceFree[token]`) and balances allocated to recipients (`recipientsClaim[user][token]`). However, no corresponding accounting adjustments are made, leading to potential failures in subsequent `claim()` calls.

Additionally, the `ownerWithdrawUnclaimed` function allows the owner to redirect any recipient's allocated balance to another address without the recipient's consent. This function sets the recipient's balance to zero and transfers the amount to a specified address.

Shell

```
function ownerWithdrawUnclaimed(
    address user,
    address token,
    address to
) external onlyOwner nonReentrant {
    require(to != address(0), "Invalid
receiver");

    uint256 amount =
recipientsClaim[user][token];
    require(amount > 0, "No balance to
withdraw");

    recipientsClaim[user][token] = 0;

    IERC20(token).safeTransfer(to, amount);
}
function sweepTokens(address token) external
onlyOwner {
    uint256 balance =
IERC20(token).balanceOf(address(this));
    require(balance > 0, "No balance");

    IERC20(token).safeTransfer(msg.sender,
balance);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

PIF - Potential Initialization Front-Run

Criticality	Medium
Location	PoolGamesNft.sol#L82 TreasuryPool.sol#L75 UserPoolGames.sol#L56
Status	Unresolved

Description

The `initialize()` functions on all three UUPS proxy contracts are publicly callable and protected only by the initializer modifier, which prevents re-initialization but does not restrict who can invoke it. If proxy deployment and initialization are performed in separate transactions, an attacker monitoring the mempool can front-run the `initialize()` call to become the owner of the proxy. This would grant the attacker full control over the contract, including upgrade authority via `_authorizeUpgrade`.

Shell

```
function initialize(address _stable) public initializer {  
    __Ownable_init(msg.sender);  
}
```

Recommendation

The team is advised to ensure that proxy deployment and initialization are performed atomically within a single transaction. This prevents any window during which an unauthorized party could call `initialize()` before the intended deployer.

AAO - Accumulated Amount Overflow

Criticality	Minor / Informative
Location	PaymentPool.sol#L25,169
Status	Unresolved

Description

The contract is using variables to accumulate values. The contract could lead to an overflow when the total value of a variable exceeds the maximum value that can be stored in that variable's data type. This can happen when an accumulated value is updated repeatedly over time, and the value grows beyond the maximum value that can be represented by the data type.

`totalRecipients` is declared as `uint8`, which has a maximum value of 255. This variable is incremented on every `addRecipient` call. When the 256th recipient is added, the increment will revert due to Solidity 0.8's checked arithmetic, producing an unhelpful error message.

```
Shell
uint8 public totalRecipients;
...
totalRecipients++;
```

Recommendation

The team is advised to carefully investigate the usage of the variables that accumulate value. A suggestion is to add checks to the code to ensure that the value of a variable does not exceed the maximum value that can be stored in its data type.

ALM - Array Length Mismatch

Criticality	Minor / Informative
Location	UserPoolGames.sol#L97
Status	Unresolved

Description

The contract is designed to handle the process of elements from arrays through functions that accept multiple arrays as input parameters. These functions are intended to iterate over the arrays, processing elements from each array in a coordinated manner. However, there are no explicit checks to verify that the lengths of these input arrays are equal. This lack of validation could lead to scenarios where the arrays have differing lengths, potentially causing out-of-bounds access if the function attempts to process beyond the end of the shorter array. Such situations could result in unexpected behavior or errors during the contract's execution, compromising its reliability and security.

Shell

```
function createUserByOwner(
    address[] memory usersOld,
    address[] memory sponsorsOld,
    uint[] memory oldValues
) external onlyOwner {
    for (uint i = 0; i < usersOld.length; i++) {
        address user = usersOld[i];
        address sponsor = sponsorsOld[i];
        uint oldValue = oldValues[i];
        ...
    }
}
```

Recommendation

To mitigate this, it is recommended to incorporate a validation check at the beginning of the function that accepts multiple arrays to ensure that the lengths of these arrays are identical. This can be achieved by implementing a conditional statement that compares the lengths of the arrays, and reverts the transaction if the lengths do not match. Such a validation step will prevent out-of-bounds errors and ensure that elements from each array are processed in a paired and coordinated manner, thus preserving the integrity and intended functionality of the contract.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	PaymentPool.sol#L39,62,161,177,190,209,219 PoolGamesNft.sol#L75,76,93,97,100,103 TreasuryPool.sol#L74,84,88,91 UserPoolGames.sol#L55,77,101,155
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

Shell

```
function _authorizeUpgrade(address newImplementation)
internal override onlyOwner {}
function setBatchProcessing(uint newValue) external
onlyOwner {}
function setManager(address managerAddress) external
onlyOwner {}
function setOldPool(address _newPool) external onlyOwner {}
function setNewPool(address _newPool) external onlyOwner {}
function setIsAuthorized(address _address, bool flag)
external onlyOwner {}
function setUser(address userAddress) external onlyOwner {}
function setBowWallet(address newAddress) external
onlyOwner {}
function setTreasuryPool(address newPool) external
onlyOwner {}
function createUserByOwner(address[] memory usersOld,
address[] memory sponsorsOld, uint[] memory oldValues)
external onlyOwner {}
function removeToken(address token) external onlyOwner {}
```

```
function addToken(address token, string calldata name)
external onlyOwner {}
function addRecipient(address newRecipient, uint24
percentage) external onlyOwner {}
function addImmutableWallet(address wallet) public
onlyOwner {}
function updateRecipientPercentage(address recipient,
uint24 newPercentage) external onlyOwner {}
function ownerWithdrawUnclaimed(address user, address
token, address to) external onlyOwner nonReentrant {}

function sweepTokens(address token) external onlyOwner {}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

DDP - Decimal Division Precision

Criticality	Minor / Informative
Location	PoolGamesNft.sol#L142,143,144,203 PaymentPool.sol#L112 TreasuryPool.sol#L170,206,271,283 UserPoolGames.sol#L176
Status	Unresolved

Description

Division of decimal (fixed point) numbers can result in rounding errors due to the way that division is implemented in Solidity. Thus, it may produce issues with precise calculations with decimal numbers.

Solidity represents decimal numbers as integers, with the decimal point implied by the number of decimal places specified in the type (e.g. decimal with 18 decimal places). When a division is performed with decimal numbers, the result is also represented as an integer, with the decimal point implied by the number of decimal places in the type. This can lead to rounding errors, as the result may not be able to be accurately represented as an integer with the specified number of decimal places.

Hence, the splitted shares will not have the exact precision and some funds may not be calculated as expected.

```
Shell
stable.safeTransfer(oldPool, (totalPrice * 50) /
100);
stable.safeTransfer(newPool, (totalPrice * 30) /
100);
stable.approve(address(feeManager), (totalPrice *
20) / 100);
uint share = valueToDistribute /
totalWeightNftToReceivePayment;
uint value = (amount * recipientsPercentage[user])
/ PRECISION;
uint profit = (amount * config.profitPercent) /
1000;
usdc.approve(address(userContract), (profit * 75)
/ 100);
((users[user][index].balance -
users[user][index].deposit) *
daysElapsed) / users[user][index].maxPeriod;
uint fee = ((half + donation.valueClaimed) * 20) /
100;

uint share = (amount * getPercentageByLevel(i)) /
100;
```

Recommendation

The team is advised to take into consideration the rounding results that are produced from the solidity calculations. The contract could calculate the subtraction of the divided funds in the last calculation in order to avoid the division rounding issue.

ERP - ERC777 Reentrancy Path

Criticality	Minor / Informative
Location	UserPoolGames.sol#L188 PoolGamesNft.sol#L289 PaymentPool.sol#L89,117
Status	Unresolved

Description

Several functions perform `safeTransfer` to untrusted addresses without reentrancy guards. If any token used by the protocol were an ERC-777 token, the recipient's `tokensReceived()` hook would execute mid-transfer, allowing reentrant calls back into the protocol. ERC-777 tokens extend ERC-20 with transfer hooks that run arbitrary code on the receiver during every transfer.

`distributeUnilevel` sends USDC to up to 20 arbitrary user addresses in a loop with no `nonReentrant` modifier. `distributePayment` sends stable tokens to arbitrary NFT owners in a batched loop, also without `nonReentrant`. Both functions mutate state between iterations, meaning a callback from any recipient could reenter and manipulate state mid-loop. `claim()` also transfers to untrusted addresses. Additionally, `PaymentPool` supports arbitrary tokens via `addToken()`, expanding the surface to any token type the owner registers.

Shell

```
usdc.safeTransfer(_user, share);
```

Recommendation

The team is advised to include reentrancy defense guards in all relevant methods as a defense-in-depth measure. This action helps prevent potential reentrancy attacks, enhancing the security and reliability of the system.

GO - Gas Optimization

Criticality	Minor / Informative
Location	PaymentPool.sol#L91
Status	Unresolved

Description

The `claim()` function inefficiently iterates over the `tokens[]` and `recipients[]` arrays. Each inner loop iteration consumes excessive gas. As a result, this inefficiency may consume gas amounts that could hinder the execution of transactions.

```
Shell
for (uint j = 0; j < recipients.length; j++) {
    address user = address(recipients[j]);
    uint value = (amount *
recipientsPercentage[user]) / PRECISION;
    recipientsClaim[user][token] += value;

    balanceFree[token] -= value;
}
```

Recommendation

The team should ensure optimal gas consumption by implementing strategies such as limiting iterations, tracking the last processed index, and using batch iterations. These measures can help reduce unnecessary computational costs, making the contract more efficient and cost-effective for users.

HV - Hardcoded Values

Criticality	Minor / Informative
Location	PoolGamesNft.sol#L117,134,142,143,144,167,169,170,171,172 TreasuryPool.sol#L170,201,206,283 UserPoolGames.sol#L170,176,259
Status	Unresolved

Description

The contract contains multiple instances where numeric values are directly hardcoded into the code logic rather than being assigned to constant variables with descriptive names. Hardcoding such values can lead to several issues, including reduced code readability, increased risk of errors during updates or maintenance, and difficulty in consistently managing values throughout the contract. Hardcoded values can obscure the intent behind the numbers, making it challenging for developers to modify or for users to understand the contract effectively.

```
Shell
stable.safeTransfer(oldPool, (totalPrice * 50) /
100);
stable.safeTransfer(newPool, (totalPrice * 30) /
100);
stable.approve(address(feeManager), (totalPrice *
20) / 100);
require(endTokenId <= 240_000, "Max supply
reached");
if (tokenId <= 100_000) return 10e6;
if (tokenId <= 150_000) return 20e6;
if (tokenId <= 200_000) return 30e6;
return 40e6;
profit = (amount * config.profitPercent) / 1000;
if (valueInPool[msg.sender] >= 100e6 && !aux.valid)
usdc.approve(address(userContract), (profit * 75) /
100);
uint fee = ((half + donation.valueClaimed) * 20) /
```

```
100;  
usdc.safeTransferFrom(msg.sender, address(this),  
    (amount * 75) / 100);  
uint share = (amount * getPercentageByLevel(i)) /  
100;  
  
return level * 100e6;
```

Recommendation

It is recommended to replace hardcoded numeric values with variables that have meaningful names. This practice improves code readability and maintainability by clearly indicating the purpose of each value, reducing the likelihood of errors during future modifications. Additionally, consider using constant variables which provide a reliable way to centralize and manage values, improving gas optimization throughout the contract.

MC - Missing Check

Criticality	Minor / Informative
Location	PoolGamesNft.sol#L91,95,98,101 TreasuryPool.sol#L80,86,89,93 UserPoolGames.sol#L79,157 PaymentPool.sol#L88,171
Status	Unresolved

Description

Several functions across the codebase accept address and value parameters without validating that they are non-zero or within expected bounds. These unchecked variables may lead to misconfiguration, opaque reverts, or unexpected behavior.

Shell

```
stable = IERC20(_stable);  
feeManager = IManager(managerAddress);  
oldPool = _newPool;  
newPool = _newPool;  
usdc = IERC20(_usdc);  
userContract = IUserPoolGames(userAddress);  
botWallet = (newAddress);  
  
treasuryPool = ITreasuryPool(newPool);
```

Setting any of these addresses to `address(0)` would cause dependent functions to revert with opaque low-level errors.

Recommendation

The team is advised to properly check the variables according to the required specifications.

MEM - Missing Error Messages

Criticality	Minor / Informative
Location	TreasuryPool.sol#L129,306 PoolGamesNft.sol#L77,78
Status	Unresolved

Description

The contract is missing error messages. Specifically, there are no error messages to accurately reflect the problem, making it difficult to identify and fix the issue. As a result, the users will not be able to find the root cause of the error.

```
Shell
require(msg.sender == address(userContract))
require(msg.sender == user || botWallet ==
msg.sender)
require(!isDistributePeriod)

require(newValue > 0)
```

Recommendation

The team is suggested to provide a descriptive message to the errors. This message can be used to provide additional context about the error that occurred or to explain why the contract execution was halted. This can be useful for debugging and for providing more information to users that interact with the contract.

MEE - Missing Events Emission

Criticality	Minor / Informative
Location	PoolGamesNft.sol#L76,93,97,100,103,107 TreasuryPool.sol#L84,88,91 UserPoolGames.sol#L77,97,155 PaymentPool.sol#L39,62,78,91,219
Status	Unresolved

Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Emitting events for significant actions is important as it allows external parties, such as wallets or dApps, to track and monitor the activity on the contract. Without these events, it may be difficult for external parties to accurately determine the current state of the contract.

Shell

```
function setBatchProcessing(uint newValue) external
onlyOwner {}
function setManager(address managerAddress) external
onlyOwner {}
function setOldPool(address _newPool) external onlyOwner {}
function setNewPool(address _newPool) external onlyOwner {}
function setIsAuthorized(address _address, bool flag)
external onlyOwner {}
function addValueToDistribute(uint amount) external {}
function setUser(address userAddress) external onlyOwner {}
function setBowWallet(address newAddress) external
onlyOwner {}
function setManager(address managerAddress) external
onlyOwner {}
function setManager(address managerAddress) external
onlyOwner {}
function setTreasuryPool(address newPool) external
onlyOwner {}
function createUserByOwner(address[] memory usersOld,
```



```
address[] memory sponsorsOld, uint[] memory oldValues)
external onlyOwner {}
function removeToken(address token) external onlyOwner {}
function addToken(address token, string calldata name)
external onlyOwner {}
function incrementBalance(uint256 amount, address
tokenContract) external nonReentrant {}
function claim() external nonReentrant {}

function sweepTokens(address token) external onlyOwner {}
```

Recommendation

It is recommended to include events in the code that are triggered each time a significant action is taking place within the contract. These events should include relevant details such as the user's address and the nature of the action taken. By doing so, the contract will be more transparent and easily auditable by external parties. It will also help prevent potential issues or disputes that may arise in the future.

PTAI - Potential Transfer Amount Inconsistency

Criticality	Minor / Informative
Location	PaymentPool.sol#L78,89 PoolGamesNft.sol#L107
Status	Unresolved

Description

The `transfer()` and `transferFrom()` functions are used to transfer a specified amount of tokens to an address. The fee or tax is an amount that is charged to the sender of an ERC20 token when tokens are transferred to another address. According to the specification, the transferred amount could potentially be less than the expected amount. This may produce inconsistency between the expected and the actual behavior.

The following example depicts the diversion between the expected and actual amount.

Tax	Amount	Expected	Actual
No Tax	100	100	100
10% Tax	100	100	90

```
Shell
stable.safeTransferFrom(msg.sender, address(this),
amount)
...
valueToDistribute += amount
```

Recommendation

The team is advised to take into consideration the actual amount that has been transferred instead of the expected.

It is important to note that an ERC20 transfer tax is not a standard feature of the ERC20 specification, and it is not universally implemented by all ERC20 contracts. Therefore, the contract could produce the actual amount by calculating the difference between the transfer call.

`Actual Transferred Amount = Balance After Transfer - Balance Before Transfer`

PTRP - Potential Transfer Revert Propagation

Criticality	Minor / Informative
Location	UserPoolGames.sol#L166
Status	Unresolved

Description

The contract sends funds to an array of recipients. A single reverting transaction may disrupt the entire flow of operations.

Shell

```
for (uint8 i = 1; i <= 20; i++) {
    uint share = (amount *
getPercentageByLevel(i)) / 100;
    address _user = levels[i - 1];
    if (_user == address(0)) {
        excess += share;
        continue;
    }

    if (
        users[_user].directs >=
getDirectsRequiredByLevel(i) &&
        treasuryPool.valueInPool(_user) >=
getPoolValueRequiredByLevel(i)
    ) {
        usdc.safeTransfer(_user, share);
        treasuryPool.increaseProfitUnilevel(_user,
share);
    } else {
        excess += share;
    }
}
```

Recommendation

The contract should tolerate the potential revert from the underlying transactions. This could be achieved by sending the funds in a non-revertable way.

SDOT - Stale Data on Transfer

Criticality	Minor / Informative
Location	PoolGamesNft.sol#L155,237 PaymentPool.sol#L39,59
Status	Unresolved

Description

The codebase contains mappings that become stale after certain operations, potentially providing inaccurate data to off-chain consumers.

In PoolGamesNft, the users mapping tracks totalWeight per address, which is incremented when NFTs are minted via `buy()` and when pending IDs are processed via `processPendingIds()`. However, when NFTs are transferred through standard ERC721 transfer functions, the totalWeight of the old and new owner is not updated.

Furthermore, in PaymentPool, the `removeToken` function removes a token from the `tokens[]` array and deletes `tokenNames[token]`, but does not clear `balanceFree[token]` or `recipientsClaim[user][token]` entries. These values remain stale in storage after removal.

Shell

```
tokens[indexToRemove] = tokens[tokens.length - 1];  
tokens.pop();  
  
delete tokenNames[token];
```

```
Shell
users[msg.sender].totalWeight++;

users[ownerOf(tokenId)].totalWeight++;
```

Recommendation

The team should consider overriding the `_update()` hook to keep the mapping in sync with ownership changes, or document that the users mapping is approximate and should not be relied upon for accurate portfolio data. In addition, the team should enhance the `removeToken` to ensure consistency across all contract operations.

SMUS - Stale Migration User State

Criticality	Minor / Informative
Location	UserPoolGames.sol#L97,124
Status	Unresolved

Description

The `createUserByOwner` function is intended to support the migration of users from a previous version of the protocol. The function accepts arrays of user addresses, their sponsors, and their old values. However, migrated users are created with `valid = false` and `directs = 0`, meaning they cannot earn unilevel commissions until they re-qualify through the normal contribution flow. The `oldValue` parameter is stored but never read by any on-chain logic.

```
Shell
function createUserByOwner(
    address[] memory usersOld,
    address[] memory sponsorsOld,
    uint[] memory oldValues
) external onlyOwner {
    ...
}
```

Recommendation

The team should consider whether migrated users should retain their qualification status from the previous system. If so, the function should accept and set `valid` and `directs` parameters accordingly.

SRC - Sybil Referral Chain

Criticality	Minor / Informative
Location	TreasuryPool.sol#L206
Status	Unresolved

Description

An attacker has the ability to generate over 20 fake accounts, known as sybil accounts, to exploit the system and claim the entire 75% unilevel commission from their own deposits. This vulnerability arises because commissions are distributed immediately upon contribution. The attacker can fulfill level requirements using this network of sybil accounts, thereby manipulating the system to their advantage.

```
Shell
usdc.approve(address(userContract), (profit * 75)
/ 100);

userContract.distributeUnilevel(msg.sender,
profit);
```

Recommendation

The team could consider a system of self-referral validation while restricting users from self-registering in the contracts.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	UserPoolGames.sol#L56,128,151 TreasuryPool.sol#L75,354 PoolGamesNft.sol#L82,97,100,103 PaymentPool.sol#L69,129
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```
Shell  
address _usdc  
address _sponsor  
address _address  
address _user  
address _stable  
address _newPool  
address _token  
  
address _wallet
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

L07 - Missing Events Arithmetic

Criticality	Minor / Informative
Location	PoolGamesNft.sol#L79
Status	Unresolved

Description

Events are a way to record and log information about changes or actions that occur within a contract. They are often used to notify external parties or clients about events that have occurred within the contract, such as the transfer of tokens or the completion of a task.

It's important to carefully design and implement the events in a contract, and to ensure that all required events are included. It's also a good idea to test the contract to ensure that all events are being properly triggered and logged.

```
Shell  
  
batchProcessing = newValue
```

Recommendation

By including all required events in the contract and thoroughly testing the contract's functionality, the contract ensures that it performs as intended and does not have any missing events that could cause issues with its arithmetic.

L16 - Validate Variable Setters

Criticality	Minor / Informative
Location	TreasuryPool.sol#L89 PoolGamesNft.sol#L98,101
Status	Unresolved

Description

The contract performs operations on variables that have been configured on user-supplied input. These variables are missing a proper check for the case where a value is zero. This can lead to problems when the contract is executed, as certain actions may not be properly handled when the value is zero.

```
Shell
botWallet = (newAddress)
oldPool = _newPool

newPool = _newPool
```

Recommendation

By adding the proper check, the contract will not allow the variables to be configured with zero value. This will ensure that the contract can handle all possible input values and avoid unexpected behavior or errors. Hence, it can help to prevent the contract from being exploited or operating unexpectedly.

L22 - Potential Locked Ether

Criticality	Minor / Informative
Location	PoolGamesNft.sol#L246,247
Status	Unresolved

Description

The contract contains Ether that has been placed into a Solidity contract and is unable to be transferred. Thus, it is impossible to access the locked Ether. This may produce a financial loss for the users that have called the payable method.

```
Shell
receive() external payable {}

fallback() external payable {}
```

Recommendation

The team is advised to either remove the payable method or add a withdrawal functionality. It is important to carefully consider the risks and potential issues associated with locked Ether.

Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
UserPoolGames	Implementation	Initializable, OwnableUpgradable, UUPSUpgradable, ReentrancyGuardUpgradable		
		Public	✓	-
	_authorizeUpgrade	Internal	✓	onlyOwner
	initialize	Public	✓	initializer
	setManager	External	✓	onlyOwner
	getUsers	External		-
	createUserByOwner	External	✓	onlyOwner
	createUser	External	✓	nonReentrant
	getUser	External		-
	setTreasuryPool	External	✓	onlyOwner
	increaseDirectMember	External	✓	onlyTreasuryPool
	setValid	External	✓	onlyTreasuryPool
	distributeUnilevel	External	✓	onlyTreasuryPool
	getPercentageByLevel	Public		-
	getDirectsRequiredByLevel	Public		-
	getPoolValueRequiredByLevel	Public		-

TreasuryPool	Implementation	Initializable, OwnableUpgr adeable, UUPSUpgra deable, ReentrancyG uardUpgrade able		
		Public	✓	-
	_authorizeUpgrade	Internal	✓	onlyOwner
	initialize	Public	✓	initializer
	setUser	External	✓	onlyOwner
	setBowWallet	External	✓	onlyOwner
	setManager	External	✓	onlyOwner
	timeUntilNextWithdrawal	Public		-
	increaseProfitUnilevel	External	✓	-
	getPlanConfig	Public		-
	_createContribution	Internal	✓	
	contribute	External	✓	nonReentrant
	getActiveContributions	External		-
	calculateDaysElapsedToClaim	Internal		
	calculateValue	Internal		
	cancelContribution	External	✓	nonReentrant
	claimContribution	External	✓	nonReentrant
	getDonation	Public		-

PoolGamesNft	Implementation	Initializable, OwnableUpgradeable, UUPSUpgradeable, ERC721Upgradeable, ERC721URIStorageUpgradeable, ReentrancyGuardUpgradeable		
		Public	✓	-
	_authorizeUpgrade	Internal	✓	onlyOwner
	setBatchProcessing	External	✓	onlyOwner
	initialize	Public	✓	initializer
	setManager	External	✓	onlyOwner
	setOldPool	External	✓	onlyOwner
	setNewPool	External	✓	onlyOwner
	setIsAuthorized	External	✓	onlyOwner
	addValueToDistribute	External	✓	-
	getNftMetadata	Public		-
	buy	External	✓	nonReentrant
	getPriceByTokenId	Public		-
	processPayments	External	✓	-
	distributePayment	Internal	✓	
	processPendingIds	Internal	✓	
		External	Payable	-
		External	Payable	-
	tokenURI	Public		-
	supportsInterface	Public		-

	getMintedTokenIdsPaginated	External		-
	getPendingTokenIdsPaginated	External		-
PaymentPool	Implementation	Ownable2Step, ReentrancyGuard		
	removeToken	External	✓	onlyOwner
	addToken	External	✓	onlyOwner
	isValidToken	Internal		
	incrementBalance	External	✓	nonReentrant
	claim	External	✓	nonReentrant
	getUserBalance	External		-
	isRecipient	External		-
	addRecipient	External	✓	onlyOwner
	addImmutableWallet	Public	✓	onlyOwner
	updateRecipientPercentage	External	✓	onlyOwner
	ownerWithdrawUnclaimed	External	✓	onlyOwner nonReentrant
	sweepTokens	External	✓	onlyOwner
IUserPoolGames	Interface			
	getUser	External		-
	increaseDirectMember	External	✓	-
	setValid	External	✓	-
	distributeUnilevel	External	✓	-

ITreasuryPool	Interface			
	valueInPool	External		-
	increaseProfitUnilevel	External	✓	-
IManager	Interface			
	incrementBalance	External	✓	-
FeeManager	Implementation	PaymentPool		
		Public	✓	Ownable

Summary

PoolGames contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io